ISA/IEC 62443 SL-3 Checklist Generator

User Manual — build product-specific 3-3 / 4-2 SL-3 testing cheat sheets

A single-file, zero-dependency companion to the ICS/OT/IoT Threat Modeler. Describe a component, add manuals / datasheets / URLs, pick any AI model, and generate a detailed, field-ready SL-3 testing cheat sheet PDF — mapped to every applicable ISA/IEC 62443-3-3 system requirement and 4-2 component requirement, with step-by-step native and external-tool test procedures.

Version 1.1.1 | Python 3.9+ | No third-party packages required

New in v1.1.0: the embedded requirement catalog and row labels now match the standards exactly (one row per requirement, no merged rows, 58 rows for an EDR / HDR / NDR sheet and 52 for a SAR sheet), every sheet gets a product-specific session setup & evidence capture row and a V&V evidence clause per row, the footer reads “Independent Assurance Services”, and a built-in QA audit checks each result. v1.1.1 refreshes the AI model lists to the providers' current model IDs. See section 12.

1. What this tool is

The **62443 SL-3 Checklist Generator** is a standalone Python program that runs a small local web app in your browser. You describe an ICS/OT/IoT product, optionally attach its manuals, datasheets, and web links, choose any AI model (with your own API key), and the tool produces a **product-specific ISA/IEC 62443-3-3 / 4-2 · SL-3 Testing Cheat Sheet** as a print-ready landscape PDF. It is the checklist companion to the ICS/OT/IoT Threat Modeler and follows the same design: one self-contained file, a model-agnostic provider router, and the same four-step workflow.

What it produces

For the confirmed component type, the sheet covers every Foundational Requirement (FR1–FR7): each applicable system requirement (SR), component requirement (CR) and component-specific EDR/HDR/NDR/SAR requirement as **its own row**. Each row gives the native configuration/verification steps (real menu paths), the specific external tools to validate that control (with exact commands or click-paths), a **V&V evidence** clause naming the screenshot, export, log or photo that proves it, the pass criteria, and product/firmware notes. The top of the sheet carries the orientation rows (where to work, interfaces/ports, firmware, **session setup & evidence capture**, how to read) and the end carries an **External Test Tools** table.

ISA/IEC 62443-3-3 / 4-2 - SL-3 TESTING CHEAT SHEET

Rockwell Automation Allen-Bradley ControlLogix 5580 (1756-L8x) - Component type: Embedded Device (EDR) - PLC/PAC - Target SL = SL 3

Where to work	Studio 5000 Logix Designer (project .ACD; Controller Properties > Security). FactoryTalk Administration Console & Security. FactoryTalk Policy Manager + System Services (CIP Security CA). Controller keyswitch RUN/REM/PROG; SD card; embedded Ethernet web page.
Interfaces / ports	EtherNet/IP explicit TCP 44818; CIP class-1 I/O UDP 2222; CIP Security 2221 (DTLS); embedded web 80/443; SNMP 161.
Firmware note	CIP Security requires controller firmware v31/32+, Studio 5000 v32+, FactoryTalk System Services (CA) + Policy Manager.
How to read	Default read-only. Each row = native how-to + external-tool validation + V&V evidence + pass criteria (Pass / Fail / Partial / **N/A** / Deferred). Write/exploit tools = isolated/representative env + written authorization.

FR 1 - Identification & Authentication Control (IAC)

Req (3-3 / 4-2)	Requirement	Where & how to verify (native steps + Ext. tools)	Pass criteria / evidence	Notes / firmware
SR/CR 1.1	Human user identification and authentication	In **Studio 5000 > Controller Properties > Security** set **Security Authority** to the FactoryTalk Security server (with **SNN**, 'bind to authority') and require authentication; assign **Permission Sets** so only authorized FT users can go online/download/edit. In **FactoryTalk Administration Console** confirm named/Windows-linked accounts (no shared). **Test:** attempt online/download as an unauthorized FT user -> denied. **Ext. tools:** FactoryTalk Admin Console (user/policy review); RSLinx/FactoryTalk Linx **Who Active** (enumerate access paths); **ReadOnlyEnum.py** (confirm no anonymous CIP). **V&V evidence:** screenshot of Controller Properties > Security and the FactoryTalk Administration Console user list.	All engineering access requires FT authentication; unauthorized users denied; no anonymous CIP path.	Needs FactoryTalk Security bound. Controller-local only = **Partial + compensating** (segmentation + CIP Security).
SR 1.6	Wireless access management	**N/A.** No wireless interface on this device; wireless access is managed at the network layer.	**N/A** - justification recorded (not assessed as a control).	**N/A**
SR/CR 1.9	Strength of public key-based authentication	In **FactoryTalk Policy Manager** confirm the CIP Security device cert uses strong keys (**ECC / RSA>=2048**) and is in-date. **Ext. tools: openssl** ('openssl s_client -connect <plc>:2221 -dtls1_2 | openssl x509 -noout -dates -issuer') or **testssl.sh** to confirm suite/expiry; **sslyze --certinfo <plc>:2221**.	Strong certificate crypto; weak/expired certs absent (openssl/testssl output attached).	**FW v32+** / CIP Security.

Layout illustration: an excerpt of page 1 rendered by the tool's built-in PDF engine from its style-example rows (ControlLogix 5580). A real sheet continues with every requirement row for FR1–FR7 and the tools table.

2. Requirements & installation

Zero dependencies

The tool uses the Python standard library only — there is nothing to pip install. The landscape PDF is produced by a built-in PDF engine; PDF/DOCX text and images are parsed with the standard library; web pages and manuals are fetched with urllib.

- **Python 3.9 or newer** (Windows, macOS, or Linux).
- A network connection to your chosen AI provider (and any URLs you scrape).
- An API key for at least one AI provider — or a local/OpenAI-compatible endpoint (Ollama, LM Studio, vLLM), which needs no key.
- **No second key.** The selected model's key does everything, including web search — there is no separate search-API key to obtain.

Run it

```
python3 ics_62443_sl3_checklist_generator.py
```

Then open the printed address (default http://127.0.0.1:8010) in your browser. Useful examples — one command per line:

```
python3 ics_62443_sl3_checklist_generator.py --port 9000 --open
python3 ics_62443_sl3_checklist_generator.py --stats
ANTHROPIC_API_KEY=sk-ant-... python3 ics_62443_sl3_checklist_generator.py
```

- **--open** — open the UI in your browser automatically.
- **--port N** — serve on a specific port (auto-tries the next few if busy).
- **--db PATH** — where saved checklists are stored (SQLite).
- **--stats** — print the embedded 62443 catalog counts and exit (v1.1.0: SR 51, CR 43, EDR 8, HDR 8, NDR 12, SAR 2).

Your API keys are stored only in your browser and are sent to the tool solely to call the provider you selected. Nothing is sent anywhere else.

3. The four-step workflow

Step 1 — Choose your AI model & key

Pick the provider and model that will write the checklist and paste that provider's API key. You can type a custom model id or point at a local endpoint. That single key does everything — including web search — so there is no second key to enter. Click **Verify & continue** and the tool validates the key.

Provider	Notes	Web search
Anthropic (Claude)	console.anthropic.com — vision-capable	Yes
OpenAI	platform.openai.com — vision-capable	Yes
Google Gemini	aistudio.google.com — vision-capable	Yes
DeepSeek	platform.deepseek.com — OpenAI-compatible	No*
Kimi (Moonshot)	OpenAI-compatible — vision-capable (kimi-k3, kimi-k2.6)	No*
Local / OpenAI-compatible	Ollama, LM Studio, vLLM — set Base URL, no key	No*

Web search runs natively through the model's own key (Anthropic web search, Google Search grounding, OpenAI web search).
*Providers without a native search tool still work — they use the model's built-in knowledge plus any URLs and files you add.

Step 2 — Describe the component

Name the product as specifically as you can — vendor, model, family, firmware/version (e.g. “Siemens SIPROTEC 5 7SJ85 overcurrent/feeder protection relay, DIGSI 5”). Add scope/context: deployment, target zone, and what the assessor can or cannot touch.

Step 3 — Add sources

- **AI web search** — tick “Let the AI search the web on its own” (on by default). The model searches the web itself, using your model key, for the product's manual, datasheet, and security guides — no separate search key.
- **Source URLs** — optionally paste product pages, online manuals, or datasheet PDFs (one per line), fetched and scraped live.
- **Uploads** — manuals and datasheets (PDF, DOCX, TXT, images). Very large manuals (400+ pages) are split into sections and fully read.

Step 4 — Confirm the type & generate

The AI first **verifies the product identity** — if your description names the wrong vendor or model (an OEM, rebranded or resold product, a legacy name, a managed vs unmanaged variant) it says so and gives the correct identity — then proposes its ISA/IEC 62443-4-2 component type. You **confirm or override** the type before generation — it drives which component-specific rows appear:

Code	Type	Typical products
EDR	Embedded Device	PLC/PAC, RTU, IED/relay, drive/VFD, meter, field instrument
HDR	Host Device	Windows/Linux server or workstation, OS-based appliance
NDR	Network Device	Switch, router, firewall, gateway, protocol converter, AP
SAR	Software Application	SCADA/HMI, historian, OPC server, engineering software

A product can be more than one type (e.g. a gateway that is EDR + NDR) — tick all that apply. **Devices with a built-in screen:** ANSI/ISA-62443-4-2 defines an embedded device (3.1.19) as special-purpose, running an embedded OS or firmware equivalent, which may have an attached control panel, and a host device (3.1.24) as a general-purpose device running an OS such as Windows or Linux that can host applications from one or more suppliers. A vendor-locked touch HMI that runs only the vendor application stays part of the EDR; the tool records a **reclassify trigger** (desktop, shell, third-party apps or extra services seen on site → add an HDR sheet for that part). The tool then generates the full sheet, streaming progress as it reads sources and maps each requirement.

4. Reading & exporting the cheat sheet

Each Foundational Requirement (FR1–FR7) is a section; each row has five columns:

- **Req (3-3 / 4-2)** — the requirement label, naming only the references that exist for the component type (e.g. SR/CR 1.1, SR 2.3, SR/EDR 2.4, CR 1.14, EDR 3.11, SR/NDR 5.2) — see section 6.
- **Requirement** — the requirement title from the standard.
- **Where & how to verify** — the native menu path and setting, an optional **Test:**, then **Ext. tools:** with the specific tools and exact commands/click-paths, then **V&V evidence:** naming the screenshot, export, log or photo that proves the row.
- **Pass criteria / evidence** — what a pass looks like, written so an assessor can adjudicate **Pass / Fail / Partial / N/A / Deferred** from the evidence.
- **Notes / firmware** — version caveats; **Partial + compensating control + residual risk** where the product can't meet a control natively; **N/A / Deferred + compensating** only where the needed access is genuinely out of scope.

Session setup & evidence capture row

Every sheet's orientation block includes a product-specific capture row: how to start a full session record and turn paging off with the product's own command (or leave it as found where paging is a persistent global setting); the product's native backup / project / configuration export with its menu path and file extension (both, when both exist); the evidence file naming convention <site>_<zone>_<assetTag>_<vendor>_<model>_<fwRev>_<YYYYMMDD-HHMM>_<initials>; SHA-256 hashing; encrypted evidence share only, never cleartext email; no device passwords saved in PuTTY/SecureCRT/tool profiles; the product actions never to perform during read-only V&V; and the offline-analysis reminder with the fields the tester writes by hand (executive-summary narrative, overall assurance level + justification, recommendations / punch list). If the model doesn't write one, a generic row is inserted and the QA audit asks you to Refine it.

Export options

- **Download PDF** — the landscape SL-3 cheat sheet matching the sample product sheets. FR headings are kept with their table header and first row, so a heading never sits alone at a page foot.
- **Printable HTML** — the same layout as an HTML page you can print to PDF.
- **Refine** — type a natural-language change (“add IEC 61850 GOOSE/MMS steps to FR3”, “treat it as EDR+NDR”) and the sheet regenerates in place.
- **Saved** — every sheet is stored locally; reopen, re-export, or delete from the Saved panel. Sheets saved with v1.0.0 are corrected (labels, rows, footer) when you open or export them.

5. How the tool chooses tools & writes steps

The value of the sheet is matching the right validation method to each requirement for the specific product — not repeating one or two tools everywhere. The tool is instructed to:

- Use the vendor's own native tools/consoles/CLIs/config files/logs first;

- Then choose appropriate open-source and commercial tools per requirement — e.g. openssl/testssl.sh for certificates; the correct protocol client (pycomm3, python-snap7, pymodbus, OPC UA, IEC 61850, DNP3) for auth/read tests; lockout and input-validation tools only in an isolated environment; CIS-CAT/OpenSCAP/Lynis for hardening baselines; passive OT monitors (Claroty, Nozomi, Dragos, Tenable.ot) for monitoring;
- Include ReadOnlyEnum.py (Isiah Jones's native-Python read-only enumeration and configuration audit tool) for read-only host/interface enumeration where it applies. ReadOnlyEnum v3 is an interactive console with no command-line options other than --no-color: start `python3 ReadOnlyEnumv3.py`, then use `target`, `use`, `scan` and `report` at its prompt;
- Research the product documentation and the web for additional, product-specific tooling — the built-in palette is a starting point, not a limit;
- Write step-by-step procedures for both the native path and each external tool, with real menu paths, settings, ports, protocols, commands, and flags.

CLI accuracy and grounding (v1.1.0)

- **One command per line** — never slash-merged (“show processes cpu sorted” and “show processes memory sorted” are two commands).
- **Verify with show/get commands**, never configuration commands; use the exact verbs of that OS family and version (IOS vs IOS-XE, Junos EX vs SRX, HiOS, FortiOS, vendor CLIs) and confirm the version first; no unfinished placeholders; known non-existent commands are banned.
- **Ground it or flag it** — anything that can't be verified from the sources is marked “confirm against <document>” or “not verified” instead of guessed.
- **Honest claims** — certifications are reported exactly as issued with their conditions; advisory status carries an as-of date and a re-check reminder; MITRE ATT&CK for ICS references give the technique ID and full name (e.g. T0831 Manipulation of Control); “MITRE EMB3D” keeps its prefix; architecture levels are plain L0–L5.

Richer sources (a real manual, a hardening guide, the vendor security advisory) produce a sharper, more accurate sheet.

6. Requirement coverage

The embedded ISA/IEC 62443 catalog drives complete coverage. The model produces one row for every applicable requirement, using N/A justifications where a control does not apply. v1.1.0 matches ANSI/ISA-62443-3-3 (2013) and ANSI/ISA-62443-4-2 (2018), which align with IEC 62443-3-3:2013 and IEC 62443-4-2:2019 in requirement numbers and titles.

Foundational Requirement	Scope
FR1 — Identification & Authentication Control	SR 1.1–1.13; CR 1.1–1.5, 1.7–1.12, 1.14; NDR 1.6 and NDR 1.13
FR2 — Use Control	SR 2.1–2.12; CR 2.1, 2.2, 2.5–2.12; EDR, HDR, NDR and SAR 2.4; EDR, HDR and NDR 2.13
FR3 — System Integrity	SR 3.1–3.9; CR 3.1, 3.3–3.9; EDR, HDR, NDR and SAR 3.2; EDR, HDR and NDR 3.10–3.14
FR4 — Data Confidentiality	SR and CR 4.1–4.3
FR5 — Restricted Data Flow	SR 5.1–5.4; CR 5.1; NDR 5.2 and NDR 5.3
FR6 — Timely Response to Events	SR and CR 6.1–6.2
FR7 — Resource Availability	SR 7.1–7.8; CR 7.1–7.4, 7.6–7.8

The tool embeds 51 system requirements, 43 base component requirements, and the EDR (8), HDR (8), NDR (12) and SAR (2) sets — no internet needed for the standard itself. A sheet has **58 rows** for an EDR, HDR or NDR and **52 rows** for a SAR.

Labels — must be 100% correct

- A row is labelled only with the references that exist for the component type: SR/CR 1.1, SR 2.3, SR/EDR 2.4, SR/SAR 3.2, CR 1.14, EDR 3.11, SR/NDR 5.2, NDR 3.10. With several types ticked, every selected type that has the requirement is named (e.g. SR/EDR/NDR 2.4).
- There is no SR 1.14. There is no base CR at 1.6, 1.13, 2.3, 2.4, 2.13, 3.2, 3.10–3.14, 5.2, 5.3, 5.4 or 7.5; never an invented prefix such as “NDR 1.1”, “SAR 3.5” or “CR 2.4”.
- SR 2.3, SR 5.4 and SR 7.5 have no component-level requirement in 62443-4-2, and SR 1.6, 1.13, 5.2 and 5.3 have one only for network devices. Those rows assess what the component contributes to the system requirement, or record N/A at the component with a justification.

No merging, ever

Every requirement is its own row with its own steps — never “1.5 / 1.7”, “6.1 / 6.2” or anything spanning FRs. If the evidence really is shared, both rows are written and each says the evidence is shared. Slash pairs of requirement numbers are avoided even in prose.

7. Built-in normalizer & QA checks

After the model writes the rows, the tool normalizes them before saving, exporting or displaying:

- Merged rows are split: the first requirement keeps the steps and each other requirement is regenerated as its own row (or, if that fails, shown with a “shared evidence” note to Refine).
- Every row is relabelled from the canonical map, filed under its correct FR in number order, and de-duplicated; rows with no valid requirement for the type are removed.

- Any requirement still missing stays visible as “(not generated — use Refine to regenerate this requirement)”.
- Markup copied from other sources is cleaned (<ip> prints as <ip>, I&A as I&A, <b> and
 become bold and line breaks), and the footer reads “Independent Assurance Services”.

The QA audit then reports, next to the result (the “check:” note) and in the progress log: invalid labels, absent or not-generated requirements, shared-evidence stubs, slash pairs of requirement numbers in text, possible slash-merged CLI commands, rows without a V&V evidence clause, a missing or generic evidence capture row, and any retired footer wording. Use **Refine** to clear them, and still review the sheet yourself before use.

8. Best results

- Give a precise product name and firmware/version — it sharpens native menu paths and version notes.
- Attach the real user manual and any hardening/security guide; add the vendor security-advisory URL if you have it.
- For 400+ page manuals, a URL to the online manual can be faster than uploading; the whole document is read either way.
- Pick a strong, current model for the deepest step-by-step detail; vision models can also read diagram images.
- Use Refine to iterate — add protocol-specific tests, expand the tools table, or re-scope the component type — until the QA audit is clear.
- Confirm the component type carefully; it determines which EDR/HDR/NDR/SAR rows appear.

9. Safety, authorization & scope

The sheets are **read-only by default**: guidance for live/production systems uses only native read-only views, show/get commands, uploads/exports and photos. Active test tooling (write, exploit, fuzz, DoS/load, brute-force, active scanning) is labelled **ISOLATED / by exception** — an isolated or representative environment with written authorization only. The sheets also tell testers:

- not to change controller modes/keyswitches, download, edit online, force, reset, or flash firmware outside a maintenance window, and never to use a vendor action that resets the device;
- to repeat the vendor's hazardous-location warnings before connecting or disconnecting USB, SD, Ethernet or modules;
- for functional-safety systems (SIS, safety controllers, protection relays), to record the safety signature and lock state, never to unlock or re-sign, and to treat bypasses, battle short/override, maintenance modes and forces as safety-significant findings;
- to hash, store and handle evidence as described in the capture row.

Confirm exact menu paths and commands against the deployed firmware and your project hardening baseline before testing, and never run intrusive tools against production control systems. The tool produces assessment guidance; it does not connect to or test your devices itself.

10. Troubleshooting

The key was rejected.

Paste the full API key with no extra spaces (Anthropic keys start with sk-ant-), confirm it is active and funded, and that the provider/model match the key. For local endpoints, set the Base URL.

A URL would not scrape.

Some vendor sites block automated fetching. Download the manual/datasheet and upload it, or paste a direct link to the PDF. The tool never bypasses site restrictions.

The port is busy.

The tool automatically tries the next few ports; or start it with --port to choose one.

A row looks generic.

Add more sources (the real manual, a hardening guide, the vendor advisory) and regenerate, or use Refine to ask for deeper native steps or specific protocol tests for that FR.

The result shows “check:” warnings.

These come from the QA audit (section 7). Use Refine on the listed rows — e.g. “write product-specific steps for SR/CR 1.7” or “add a V&V evidence clause to every FR2 row”.

An old sheet changed when I opened it.

Sheets saved with v1.0.0 are normalized to the v1.1.0 rules when opened or exported (labels, row split, footer). Requirements v1.0.0 never produced (e.g. EDR 2.4, HDR 2.13, NDR 2.13) show as “not generated” — use Refine to fill them.

Can I run it fully offline?

Yes, with a local model (Ollama/LM Studio/vLLM). The 62443 catalog and PDF engine are built in; only web scraping/search and remote AI providers need a connection.

Where are my saved checklists?

In a local SQLite database (default in your home folder; change with --db). Nothing is uploaded to any external service.

11. At a glance

Item	Value
Run	python3 ics_62443_sl3_checklist_generator.py
Default URL	http://127.0.0.1:8010
Dependencies	None (Python 3.9+ standard library only)
AI providers	Anthropic, OpenAI, Gemini, DeepSeek, Kimi, local/OpenAI-compatible
Web search	Built-in — the AI model searches with your key (Anthropic, OpenAI, Google)
Component types	EDR, HDR, NDR, SAR (and combinations)
Catalog	51 SR, 43 CR, EDR 8, HDR 8, NDR 12, SAR 2 — 58 rows (EDR/HDR/NDR) or 52 rows (SAR)
Output	Landscape SL-3 cheat-sheet PDF + printable HTML; saved locally; QA audit with every result

12. What changed in v1.1.0 and v1.1.1

Area	Correction / improvement
Requirement catalog	Removed base CRs that don't exist in 62443-4-2 (1.6, 2.3, 2.4, 3.2, 3.10–3.14, 5.2, 5.3); added EDR 2.4, HDR 2.13 and NDR 2.13; titles aligned with the standards (e.g. CR 1.9 “Strength of public key-based authentication”).
Labels & rows	One row per requirement number with only existing references (SR 2.3 and SR 7.5, not SR/CR; SR/EDR 2.4 instead of separate SR/CR 2.4 and EDR rows). EDR/HDR/NDR sheets now have 58 rows, SAR sheets 52.
No merging	Merged rows are split and the missing requirement regenerated; rows re-filed under the right FR and de-duplicated; v1.0.0 prompt example “SR/CR 1.13 (+ NDR 1.13)” removed.
Depth standard	V&V evidence clause per row; Pass / Fail / Partial / N/A / Deferred; Partial + compensating + residual risk; N/A / Deferred + compensating only when out of scope.
Evidence capture	Product-specific session setup & evidence capture row (logged session, native backup, naming, SHA-256, handling, never-do list, manual report fields).
Safety & accuracy	Read-only default, ISOLATED / by-exception labelling, controller and functional-safety don'ts, CLI accuracy rules, ground-or-flag, identity check, EDR vs HDR rule, honest certification and dated advisories.
Footer	“Independent Assurance Services” (the v1.0.0 “Independent Assurance Authority” wording is replaced, including in saved sheets).
PDF / HTML	FR headings kept with their table header and first row; reportlab/HTML markup and entities cleaned; print CSS keeps headings and rows together.
QA	Built-in normalizer and QA audit shown with every result and on saved sheets.
Manual	Coverage table corrected (v1.0.0 listed CR 1.6/1.13, CR 2.13, CR 3.10–3.14 and “54 component requirements”, and printed “&”); run examples one per line; layout illustration added in place of the “[sample]” placeholder.

Area	Correction / improvement
v1.1.1 — model lists	Provider model lists updated to current API model IDs (checked 2 Oct 2026): Anthropic claude-opus-5-5, claude-sonnet-5-5, claude-haiku-4-5-20251001, claude-fable-5-1; OpenAI gpt-6-astra, gpt-6.1-sol, gpt-6-luna; Google gemini-3.8-flash, gemini-3.1-pro-preview, gemini-3.5-flash-lite; DeepSeek deepseek-v4-pro, deepseek-flash; Kimi kimi-k3, kimi-k2.6. Retired IDs removed (e.g. deepseek-chat / deepseek-reasoner, discontinued 24 Jul 2026; moonshot-v1-*). You can still type any model id.
v1.1.1 — ReadOnlyEnum	Described as Isiah Jones's native-Python read-only enumeration and configuration audit tool (v1.1.0 called it an “asset-owner tool”).
v1.1.1 — Kimi images	The Kimi (Moonshot) provider is now vision-capable, so uploaded images and images extracted from PDFs are sent to kimi-k3 / kimi-k2.6 (both accept images).
v1.1.1 — ReadOnlyEnum usage	The built-in example and tool palette now use ReadOnlyEnum v3's real interactive workflow (set project, target, use, scan, results, report). v1.0.0/v1.1.0 showed a “--target” flag that ReadOnlyEnum does not have.